

Flask Framework

Complete Study Notes with Original Diagrams — Python Web Development

1. What is Flask?

Flask is a **web framework written in Python**. It provides tools, libraries, and technologies that allow you to build web applications and APIs quickly and easily. Flask is called a **micro-framework** because it keeps the core simple but is extendable.

Flask was built using two main technologies:

- ① **WSGI** — Web Server Gateway Interface
- ② **Jinja2** — Template Engine

Your Original Notes — Page 1 (Overview & WSGI Diagram):

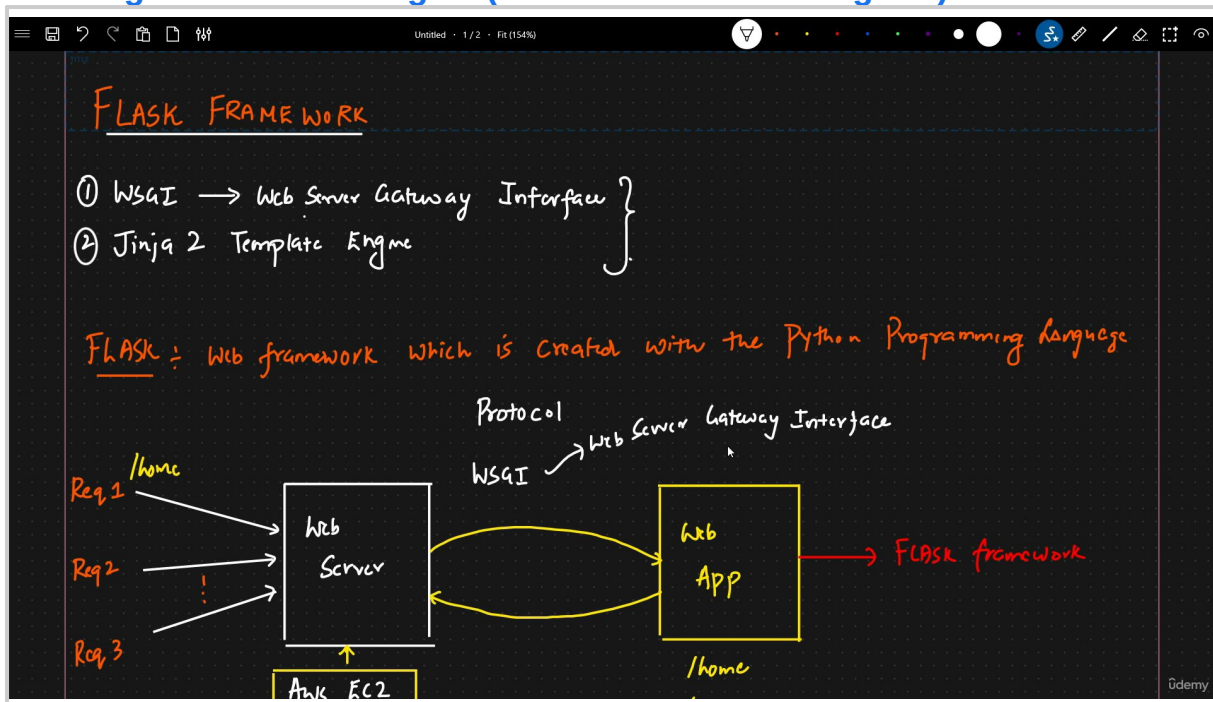


Image 1 — Flask Framework overview: WSGI, Jinja2, and the Web Server → WSGI → Web App flow

2. Why Should We Use Flask?

Reason	Explanation
Simple & Lightweight	Very easy to start — just a few lines of Python code
Python-based	If you know Python, you already know most of Flask
Flexible	You choose what to add — no unnecessary bloat
Great for ML Projects	Data scientists use Flask to deploy ML models as web apps

Large Community

Tons of tutorials, plugins, and help available online

3. Concept 1 — WSGI (Web Server Gateway Interface)

WSGI is a **protocol** — a set of rules that allows a Web Server to communicate with a Web Application (like Flask). Think of it as a **bridge or translator** between the web server and your Flask app.

How It Works — Step by Step:

Step	What Happens
1	User opens a browser and types a URL (e.g. /home)
2	Request goes to the Web Server (AWS EC2 / Apache / IIS / Azure)
3	Web Server uses WSGI to pass the request to Flask (Web App)
4	Flask processes the request and sends a response back via WSGI
5	User sees the result in their browser

Your Original Notes — Page 1 (Full WSGI Diagram with Server Options):

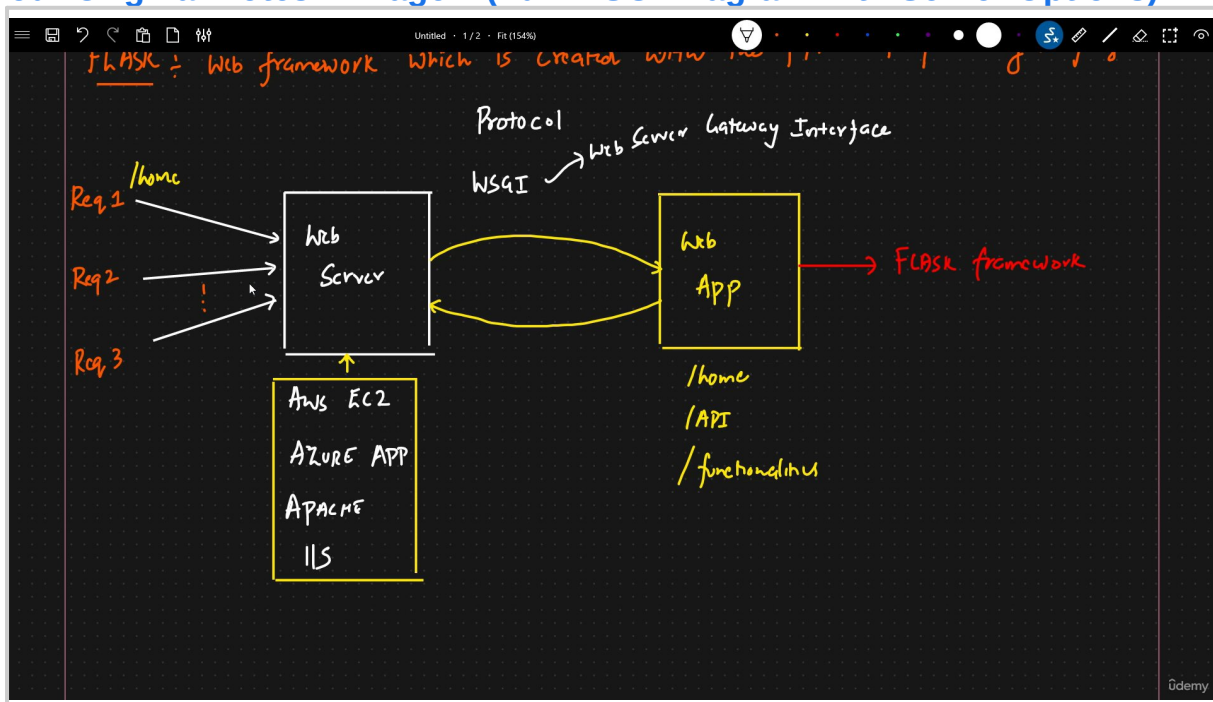


Image 2 — Full WSGI diagram: Requests → Web Server (AWS EC2 / Azure / Apache / IIS) → WSGI → Web App (Flask) with route examples /home /API /functionlinus

Diagram Explained:

In your diagram: **Req 1, Req 2, Req 3** are user requests from the internet. They hit the **Web Server** first. The yellow oval between the Web Server and the Web App box represents the **WSGI connection**. The yellow Web App box is your **Flask application**.

Web Server Platforms (from your notes):

Platform	Type
AWS EC2	Amazon Cloud Server
Azure App	Microsoft Cloud Server
Apache	Open Source Web Server
IIS	Microsoft Internet Information Services

Flask URL Routes (from your notes):

- **/home** — The homepage of your website
- **/API** — An endpoint to serve data
- **/functionlinus** — A custom function endpoint

4. Concept 2 — Jinja2 Template Engine

Jinja2 is a **Web Template Engine** used by Flask to create **dynamic web pages**. Instead of writing separate HTML for every user, Jinja2 fills in a template with real data from a source — making the page different for each user or situation.

Your Original Notes — Page 2 (Jinja2 Diagram):

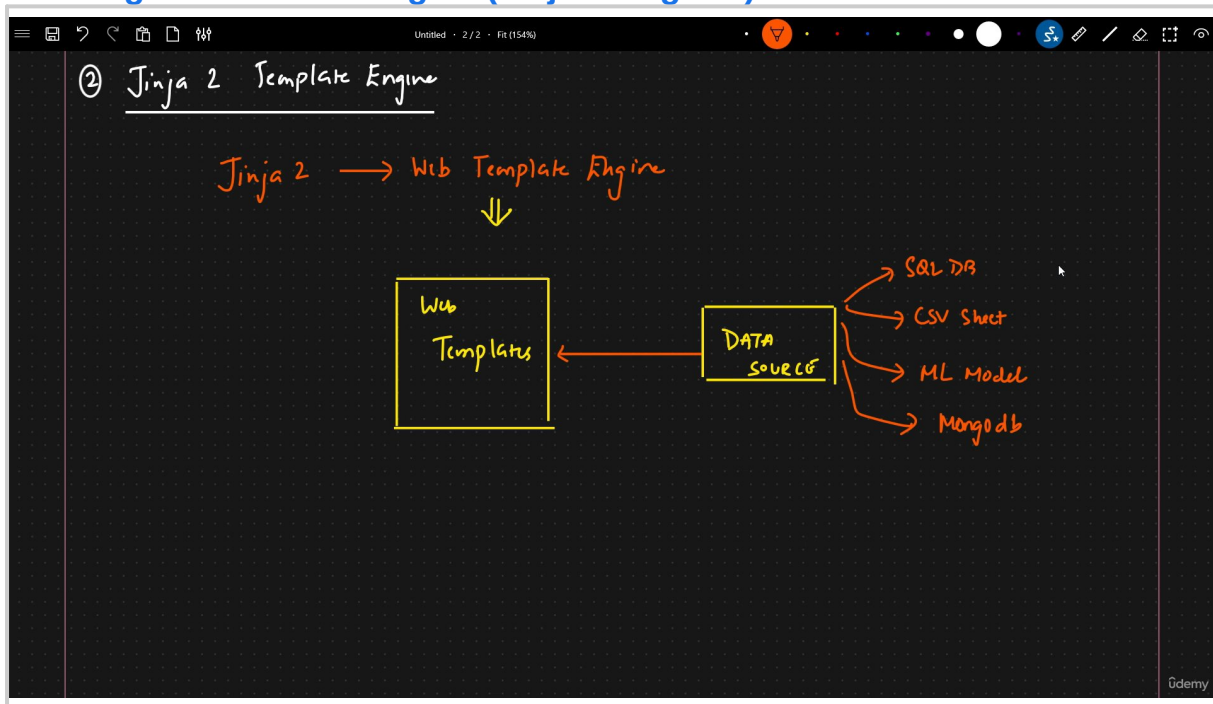


Image 3 — Jinja2 Template Engine: Data Sources (SQL DB, CSV Sheet, ML Model, MongoDB) flow into Web Templates to generate dynamic pages

Static vs Dynamic Pages:

Type	Meaning	Example
Static Page	Same HTML shown to everyone	A basic "About Us" page
Dynamic Page	HTML changes based on data	Login page showing YOUR name

Supported Data Sources (from your notes):

Data Source	Description
SQL Database	Relational database (MySQL, PostgreSQL, SQLite)
CSV Sheet	Spreadsheet data files
ML Model	Machine Learning model predictions/results
MongoDB	NoSQL / document-based database

5. Creating Dynamic Web Pages

Your Image 4 shows two practical examples of how Flask + Jinja2 create dynamic web pages:

Your Original Notes — Page 2 (Dynamic Web Pages Examples):

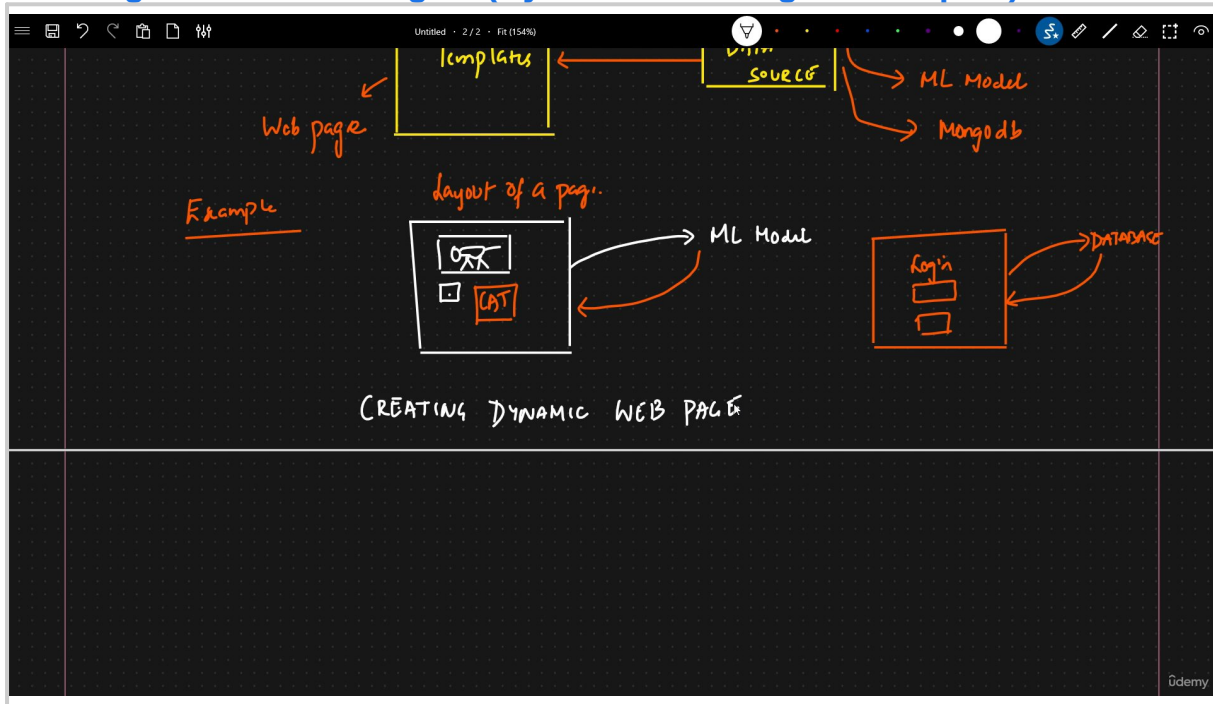


Image 4 — Creating Dynamic Web Pages: (Left) ML image classifier web page, (Right) Login form connected to a database

Example 1 — Cat Image Classifier (ML App):

- User uploads an image on the web page
- Flask sends the image to an **ML Model**
- ML Model predicts: 'It is a CAT'
- Jinja2 injects that result into the HTML template
- User sees the result page — generated **dynamically!**

Example 2 — Login Page (Database App):

- User fills in username and password in the login form
- Flask checks credentials against a **Database**
- Jinja2 renders a personalized welcome page for that user
- Each user sees a different page — that is dynamic web!

6. The Big Picture — How Everything Connects

Here is the full flow of a Flask web application from start to finish:

Step	Component	Action
1	User / Browser	Types a URL in the browser (e.g. /home)
2	Web Server	AWS EC2 / Apache receives the HTTP request

3	WSGI	Protocol passes the request to Flask app
4	Flask App	Python code processes the request/logic
5	Jinja2 + Data Source	Fetches data from DB/ML Model and fills template
6	HTML Response	Dynamic page sent back to user via WSGI
7	User / Browser	User sees the final web page

7. Quick Reference Summary

Term	Simple Meaning
Flask	Python tool/framework to build websites and web apps
WSGI	Bridge/protocol between Web Server and Flask app
Web Server	AWS EC2, Apache etc. — receives all user requests
Jinja2	Fills HTML templates with real data — makes pages dynamic
/home, /API	Routes — different URL pages in your Flask app
Data Source	Where Flask gets data: SQL DB, CSV, ML Model, MongoDB
Dynamic Page	Web page that changes based on user or data

8. Real-Life Example — ML Model as a Web App

Imagine you train a **cat vs dog image classifier** in Python. Here is how Flask turns it into a live website anyone can use:

- 1 — User visits your website and uploads an image
- 2 — Flask receives it through the /predict route
- 3 — Flask passes the image to your Python ML Model
- 4 — ML Model returns: 'CAT — 97% confidence'
- 5 — Jinja2 puts that result into an HTML results template
- 6 — User sees a beautiful result page — all dynamically generated!

This is exactly what your Image 4 diagram was showing! Flask is the most popular tool in Data Science and ML for deploying Python models as real web applications.